🏛️ 1. The Core Philosophy: "The Dialectic"

The project is rooted in the **Hegelian Dialectic** (Thesis + Antithesis = Synthesis).

- **The Problem:** Standard AI models often "echo" the bias of the input or hallucinate details to sound more confident.
- **The Solution:** Instead of one AI writing a summary, **Dialectic AI** forces two AIs into an adversarial debate. One defends the text (**Supporter**), while the other challenges it (**Challenger**). Their conflict uncovers the objective truth.

🏗️ 2. The Architectural Engine (LangGraph)

The project is built on **LangGraph**, a state-of-the-art framework for building cyclic, stateful multi-agent systems. Unlike a simple "prompt chain," this is a **State Machine** that tracks variables across different "Nodes."

The Workflow Pipeline:

1. **The Analyst Node:** The orchestrator. It reads the article and dynamically decides which two "Personas" should debate (e.g., if it's a tech article, it might assign a "Privacy Advocate" vs. a "Silicon Valley CEO").
2. **Adversarial Nodes:** The Challenger and Supporter generate independent summaries based on their assigned personas.
3. **The Fact-Checker Node:** An algorithmic (non-LLM) node that performs deep linguistic analysis.
4. **The Fallacy Detector:** A "Reflexion" node where the LLM reviews the debate for logical errors (Chain-of-Thought).
5. **The Mediator Node:** The final synthesis node that takes the best of both arguments and produces a neutral, grounded report.

🔍 3. Advanced RAG & Multi-Source Grounding

This project implements a sophisticated **Retrieval-Augmented Generation (RAG)** pipeline that goes far beyond simple document search:

- **URL-to-Text RAG:** The system automatically detects if the input is a URL. It then fetches the live HTML, strips scripts/CSS, and extracts the core content.
- **Entity-Level Web RAG:** While most RAG systems pull data *before* generation, Dialectic AI performs **Just-In-Time RAG** *during* the evaluation phase.
    - If the **Fact-Checker** finds a Named Entity (e.g., "Apple Inc.") that wasn't in the original text, it doesn't immediately flag it as a hallucination.
    - It triggers a **DuckDuckGo Web Search** to see if that entity is a real-world fact that the model brought in from its own knowledge.

- **Result:** It distinguishes between "Knowledgeable Context" and "Hallucinations."

🧪 4. The Technical Stack (Deep Dive)

- **Inference Engine: Groq Cloud API**. This allows the agents to "think" at 300+ tokens per second using **Llama 3.3-70b** (Reasoning) and **Qwen-32b** (Efficiency).
- **Linguistic analysis: spaCy (en_core_web_md)**. Used for high-precision Named Entity Recognition (NER) to track People, Organizations, and Locations.
- **Semantic Verification (NLI): DeBERTa-v3-small Cross-Encoder**. The system uses Natural Language Inference to calculate the probability of "Contradiction" between a summary sentence and the original text.
- **Stochastic Scoring:** Uses **NumPy** to calculate a final "Authority Score" based on entity overlap and NLI contradiction penalties.

📊 5. Evaluation & Quantified Truth

The project provides a **Quantitative Authority Score (0.0 - 1.0)** for every summary.

- **Mathematical Grounding:** Score = (Verified Entities / Total Entities) * (1 - Contradiction Penalty).
- **Visual Hallucination Highlighting:**
  - 🟢 **Green:** Verified against the source text.
  - 🔵 **Blue:** Verified via External Web RAG (DuckDuckGo).
  - 🔴 **Red:** Unverified/Hallucinated - the system warns the user.

💻 6. UI/UX: The Neutrality Dashboard

Built with **Streamlit**, the interface is designed for professional transparency:

- **3-Column layout:** Allows the user to watch the "Trial" happen in real-time.
- **Persona Identification:** Shows you exactly which experts are debating.
- **Self-Correction Logic:** If a summary is factually poor, the system **automatically loops back** and forces the agent to rewrite (The router_logic in graph.py).

🧬 7. Project Structure (A-Z)

- **.env**: Stores Groq/Search keys.
- **agents/**: The "Brains" (Analyst, Challenger, Supporter, Mediator).
- **graph.py**: The "Central Nervous System" (Routing and Workflow).

- **state.py**: The "Shared Memory" between agents.
- **config.py**: Model loading and system suppressions.
- **utils/tools.py**: The "Hands" (Web search and HTML scraping).
- **app.py**: The UI gateway.

This is not just a summarizer; it is a **truth-verification engine** that uses AI to check AI.